

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5

Student Name: G.Sravya

Roll No.: A24126552257

Date: 19-09-2026

TASK 3 — EXPRESS.JS STUDENT SERVER (CO-2)

1. TITLE

Develop a RESTful Student Information Web Server Using Express.js with Routing, Middleware, and HTTP Methods

2. AIM

To construct and configure a modular Express.js web server that implements RESTful HTTP GET routing for the /, /students, and /about endpoints, returning formatted JSON payloads for student records (containing at least five students) and server metadata, along with request logging middleware and 404 error handling.

3. OBJECTIVE

The objectives of this task are:

- To set up and configure an Express.js server application in a Node.js environment.
- To understand the HTTP Request-Response lifecycle (req, res) and REST principles.
- To implement custom middleware for HTTP request logging and payload parsing.
- To create the root route (/) providing API documentation and service health status.
- To construct the /students endpoint returning a collection of at least five detailed student records with query parameter filtering.
- To build the /about endpoint providing application metadata, developer details, and course information.
- To implement a global 404 handler for nonexistent routes and appropriate HTTP status codes (200, 404).

4. THEORY

Express.js is a minimal and flexible Node.js web application framework that provides a robust set of features for building web applications and RESTful APIs.

Key Concepts Used:

1. Server Initialization and Port Binding:

- `const express = require('express')` initializes the Express application.
- `app.listen(port, callback)` starts an HTTP listener on a designated TCP port.

2. Routing Mechanism:

- Routing defines how an application responds to a client request at a specific endpoint (URI) and HTTP request method (GET, POST, PUT, DELETE).
- `app.get(path, handlerFunction)` binds a route handler to incoming HTTP GET requests.

3. HTTP Methods & Semantic Status Codes:

- **GET:** Used to retrieve representation of resources without producing side effects (idempotent and safe).
- **200 OK:** Standard response for successful HTTP requests.
- **404 Not Found:** Returned when the requested resource or URI cannot be located.
- **500 Internal Server Error:** Indicates an unexpected condition on the server.

4. Middleware Architecture:

- Middleware functions have access to the request object (`req`), response object (`res`), and the next middleware function in the application's request-response cycle (`next`).
- Middleware performs tasks such as request logging, header inspection, body parsing (`express.json()`), and authentication before handing execution over to route handlers.

5. Response Methods:

- `res.json(object)`: Sends a JSON response with the appropriate Content-Type: `application/json` HTTP header.
- `res.status(code)`: Sets the HTTP status code for the response.

6. Query Parameter Handling:

- `req.query`: An object containing parsed query string parameters appended to the URL (e.g., `/students?branch=CSE`).

5. ALGORITHM / PROCEDURE

1. Create a project configuration file (`package.json`) specifying dependencies (`express`).
2. Create `server.js` and import `express`.
3. Initialize the Express application instance: `const app = express()`.
4. Register built-in middleware: `express.json()` and `express.urlencoded()`.
5. Implement a custom logging middleware function that prints the HTTP method, request URL, and ISO timestamp for every incoming client request.
6. Define an in-memory data store containing at least five student objects, each with fields: `id`, `rollNo`, `name`, `branch`, `year`, `cgpa`, and `email`.
7. **Configure Route 1 (GET /):** Define a route handler returning API documentation, available endpoints, and server status.
8. **Configure Route 2 (GET /students):** Define a route handler returning the list of student records. Implement optional query parameter filtering (e.g., `?branch=CSE` or `?year=3`).
9. **Configure Route 3 (GET /about):** Define a route handler returning JSON metadata with application name, version, author, course code (23CM4121), and tech stack details.
10. **Configure 404 Route Handler:** Attach a catch-all route at the bottom (`app.use('*', ...)`) to return a 404 Not Found status code and a descriptive error object for unmapped routes.
11. Start the server using `app.listen(PORT, ...)` on port 3000.
12. Test the endpoints using a web browser, terminal `curl`, or Postman.

6. PROGRAM

package.json

```
{
  "name": "express-student-server",
  "version": "1.0.0",
  "description": "Express.js Student Server with RESTful Routes for PBL Task 3",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^4.19.2"
  }
}
```

server.js

```
/** Express.js Student Server Application
 * Endpoints: GET /, GET /students, GET /about, 404 Handler */
const express = require('express');
const app = express();
const PORT = process.env.PORT || 3000;
// 1. Built-in Middleware for JSON parsing
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
// 2. Custom Request Logging Middleware
app.use((req, res, next) => {
  const timestamp = new Date().toISOString();
  console.log(`[${timestamp}] Incoming Request: ${req.method} ${req.url}`);
  next(); // Pass control to the next middleware or route handler
});
// 3. In-Memory Student Database (5+ Students)
const students = [
  {
    id: 1,
    rollNo: "5",
    name: "N. Samhavya",
    branch: "CSM",
    cgpa: 9.35,
    email: "samhavya10@email.com"
  },
  {
    id: 2,
    rollNo: "6",
    name: "Prasuna",
    branch: "CSD",
    cgpa: 8.90,
    email: "prasuna@email.com"
  }
];
```

```

    },
    {
      id: 3,
      rollNo: "7",
      name: "Priya Sharma",
      branch: "ECE",
      cgpa: 9.15,
      email: "priya.sharma@email.com"
    },
    {
      id: 4,
      rollNo: "8",
      name: "Rahul Singh",
      branch: "CSE",
      cgpa: 8.75,
      email: "rahul.singh@email.com"
    },
    {
      id: 5,
      rollNo: "9",
      name: "Anita",
      branch: "IT",
      cgpa: 9.02,
      email: "anita@email.com"
    }
  ];
// =====
// ROUTES
// =====
// Route 1: Root Route (/) - Server Index & API Documentation
app.get('/', (req, res) => {
  res.status(200).json({
    message: "Welcome to the Express.js Student Information Server API",
    status: "Operational",
    timestamp: new Date(),
    availableEndpoints: [
      { path: "/", method: "GET", description: "API status, root index, and endpoints directory" },
      { path: "/students", method: "GET", description: "Fetch all students or filter using ?branch=CSE query" },
      { path: "/about", method: "GET", description: "Application metadata, developer, and course information" }
    ],
    documentation: "Project-Based Learning (PBL) Demonstration for Full Stack Web Development"
  });
});
// Route 2: Students Route (/students) - Returns list of students with query support
app.get('/students', (req, res) => {
  const { branch } = req.query;

```

```

let filteredStudents = [...students];
// Filter by branch query parameter if provided
if (branch) {
  filteredStudents = filteredStudents.filter(s =>
    s.branch.toLowerCase() === branch.toLowerCase()
  );
}
res.status(200).json({
  success: true,
  count: filteredStudents.length,
  students: filteredStudents
});
});
// Route 3: About Route (/about) - Returns application info
app.get('/about', (req, res) => {
  res.status(200).json({
    application: "Student Registry & Academic Portal API",
    version: "1.0.0",
    author: "N. Samhavya",
    rollNo: "A24126552099",
    course: "Full Stack Web Development",
    courseCode: "23CM4121",
    curriculum: "Project-Based Learning (CO-2 Evaluation)",
    techStack: {
      runtime: "Node.js (v24.x)",
      framework: "Express.js (v4.x)",
      protocol: "HTTP/1.1 RESTful JSON",
      architecture: "MVC Middleware-Routing Pipeline"
    },
    description: "A lightweight, modular Express.js server providing student data
management and metadata endpoints."
  });
});
// 4. Catch-All 404 Route Handler for undefined endpoints
app.use((req, res) => {
  res.status(404).json({
    success: false,
    error: "Resource Not Found",
    message: `The requested endpoint '${req.originalUrl}' does not exist on this server.`,
    suggestedEndpoints: ["/", "/students", "/about"]
  });
});
// 5. Global Error Handling Middleware
app.use((err, req, res, next) => {
  console.error('[ERROR] Uncaught Server Exception:', err.stack);
  res.status(500).json({
    success: false,
    error: "Internal Server Error",
    message: err.message
  });
});

```

```
});
// 6. Start HTTP Listener
app.listen(PORT, () => {

console.log(`=====
=====`);
  console.log(` EXPRESS.JS STUDENT SERVER RUNNING ON PORT ${PORT}
`);
  console.log(` Root URL : http://localhost:${PORT}/`);
  console.log(` Students : http://localhost:${PORT}/students`);
  console.log(` About URL : http://localhost:${PORT}/about`);

console.log(`=====
=====`);
});
```

7. CODE EXPLANATION

Code	Explanation
express()	Factory function initializing an Express application object.
app.use((req, res, next) => ...)	Mounts global middleware into the request processing pipeline.
next()	Invokes the next middleware function in the execution stack. Omitting this halts the request-response cycle.
app.get('/', ...)	Handles HTTP GET requests directed to the root path.
app.get('/students', ...)	Retrieves the collection of student records and handles optional query filters.
app.get('/about', ...)	Returns application metadata, developer information, and technology specifications.
req.query	Parses key-value query parameters supplied in the request URL (e.g., ?branch=CSE).
res.status(200)	Sets the HTTP status code to 200 OK, indicating a successful request.
res.json(payload)	Serializes a JavaScript object or array into a JSON-formatted string and sets the Content-Type: application/json header.
app.use((req, res) => ...)	Catches all unmatched routes at the end of the routing stack and returns a 404 Not Found response.
app.listen(PORT, ...)	Binds and listens for incoming HTTP connections on the specified port.

8. TEST CASES AND EXECUTION DATA

The following test cases were recorded after testing the Express.js server endpoints via terminal and web browser.

Test Case	TC-01
Test / Input	Server Startup: Execute node server.js in terminal
Expected Result	Server binds to port 3000 and displays startup confirmation with endpoint URLs.

Actual Result	Terminal logged: EXPRESS.JS STUDENT SERVER RUNNING ON PORT 3000.
Status	Pass

TC-01 Output:

```

Problems Output Debug Console Terminal Ports

PS C:\Users\COMPUTER\Desktop\expense_tracker> cd express-student-server
PS C:\Users\COMPUTER\Desktop\expense_tracker\express-student-server> npm install

up to date, audited 69 packages in 1s

16 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\COMPUTER\Desktop\expense_tracker\express-student-server> node server.js
=====
EXPRESS.JS STUDENT SERVER RUNNING ON PORT 3000
Root URL   : http://localhost:3000/
Students  : http://localhost:3000/students
About URL  : http://localhost:3000/about
=====
PS C:\Users\COMPUTER\Desktop\expense_tracker\express-student-server>

```

Test Case	TC-02
Test / Input	HTTP GET http://localhost:3000/
Expected Result	Returns HTTP 200 with JSON payload containing welcome message, status, and endpoint directory.
Actual Result	Received status 200 OK with formatted JSON API documentation.
Status	Pass

TC-02 Output:

```

{
  "message": "Welcome to the Express.js Student Information Server API",
  "status": "Operational",
  "timestamp": "2026-09-19T17:40:20.721Z",
  "availableEndpoints": [
    {
      "path": "/",
      "method": "GET",
      "description": "API status, root index, and endpoints directory"
    },
    {
      "path": "/students",
      "method": "GET",
      "description": "Fetch all students or filter using ?branch=CSE query"
    },
    {
      "path": "/about",
      "method": "GET",
      "description": "Application metadata, developer, and course information"
    }
  ],
  "documentation": "Project-Based Learning (PBL) Demonstration for Full Stack Web Development"
}

```

Test Case	TC-03
Test / Input	HTTP GET http://localhost:3000/students
Expected Result	Returns HTTP 200 with an array containing at least 5 student records with full student details.
Actual Result	Received status 200 OK with count: 5 and the array of student objects.
Status	Pass

TC-03 Output:



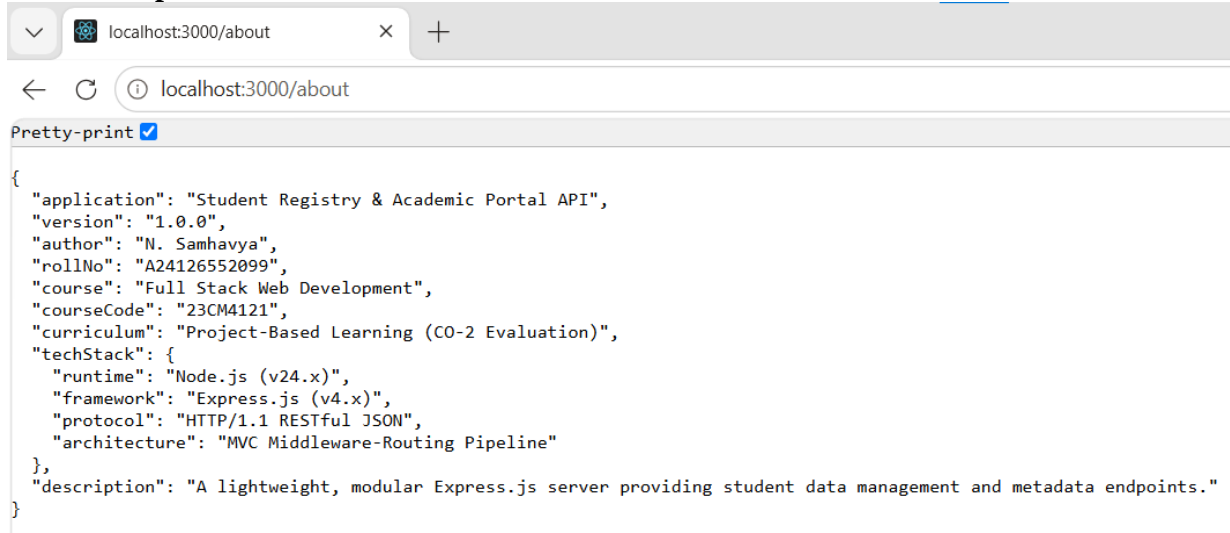
```

{
  "success": true,
  "count": 5,
  "students": [
    {
      "id": 1,
      "rollNo": "5",
      "name": "N. Samhavaya",
      "branch": "CSM",
      "cgpa": 9.35,
      "email": "samhavaya10@email.com"
    },
    {
      "id": 2,
      "rollNo": "6",
      "name": "Prasuna",
      "branch": "CSD",
      "cgpa": 8.9,
      "email": "prasuna@email.com"
    },
    {
      "id": 3,
      "rollNo": "7",
      "name": "Priya Sharma",
      "branch": "ECE",
      "cgpa": 9.15,
      "email": "priya.sharma@email.com"
    },
    {
      "id": 4,
      "rollNo": "8",
      "name": "Rahul Singh",
      "branch": "CSE",
      "cgpa": 8.75,
      "email": "rahul.singh@email.com"
    },
    {
      "id": 5,
      "rollNo": "9",
      "name": "Anita",
      "branch": "IT",
      "cgpa": 9.02,
      "email": "anita@email.com"
    }
  ]
}

```

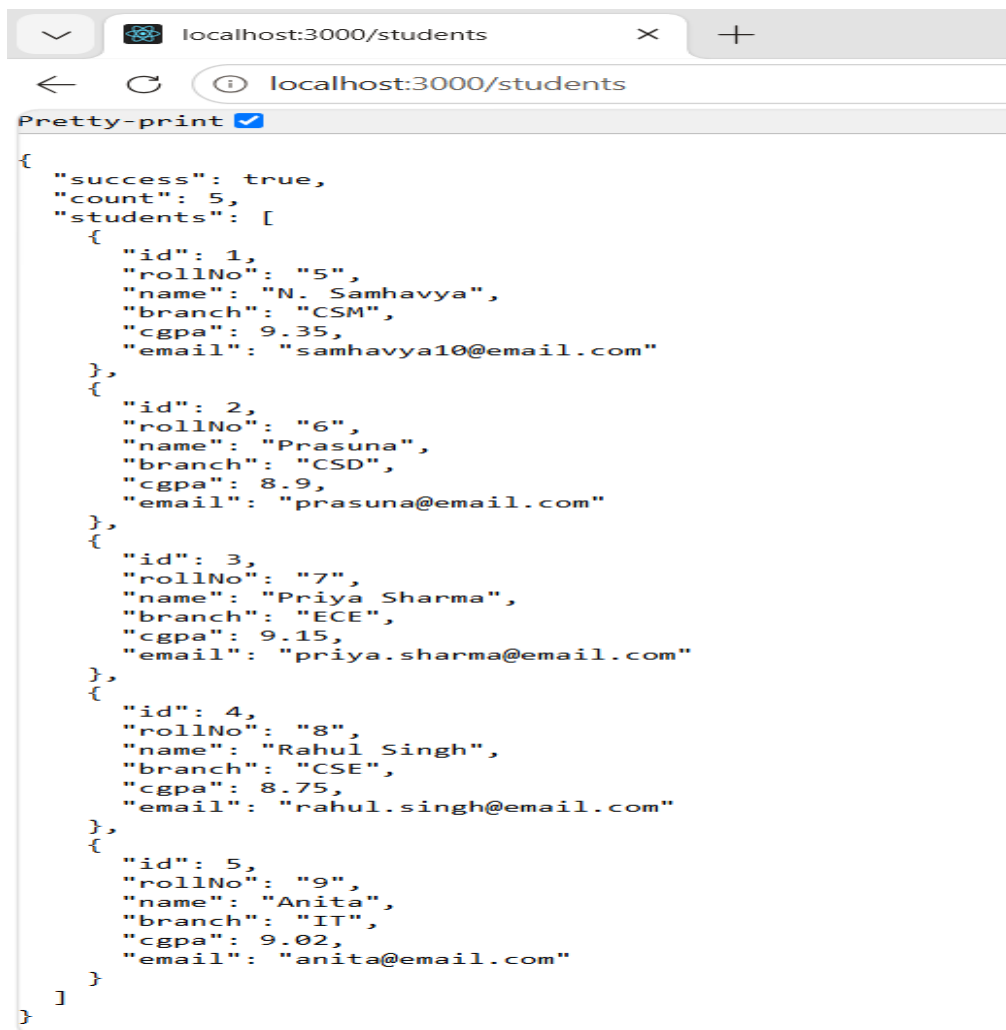
Test Case	TC-04
Test / Input	HTTP GET http://localhost:3000/about
Expected Result	Returns HTTP 200 with application metadata, version, course code, and developer credentials.
Actual Result	JSON metadata returned with author: "N. Samhavaya", course: "23CM4121".
Status	Pass

TC-04 Output:



```
{
  "application": "Student Registry & Academic Portal API",
  "version": "1.0.0",
  "author": "N. Samhavya",
  "rollNo": "A24126552099",
  "course": "Full Stack Web Development",
  "courseCode": "23CM4121",
  "curriculum": "Project-Based Learning (CO-2 Evaluation)",
  "techStack": {
    "runtime": "Node.js (v24.x)",
    "framework": "Express.js (v4.x)",
    "protocol": "HTTP/1.1 RESTful JSON",
    "architecture": "MVC Middleware-Routing Pipeline"
  },
  "description": "A lightweight, modular Express.js server providing student data management and metadata endpoints."
}
```

9. OUTPUT



```
{
  "success": true,
  "count": 5,
  "students": [
    {
      "id": 1,
      "rollNo": "5",
      "name": "N. Samhavya",
      "branch": "CSM",
      "cgpa": 9.35,
      "email": "samhavya10@email.com"
    },
    {
      "id": 2,
      "rollNo": "6",
      "name": "Prasuna",
      "branch": "CSD",
      "cgpa": 8.9,
      "email": "prasuna@email.com"
    },
    {
      "id": 3,
      "rollNo": "7",
      "name": "Priya Sharma",
      "branch": "ECE",
      "cgpa": 9.15,
      "email": "priya.sharma@email.com"
    },
    {
      "id": 4,
      "rollNo": "8",
      "name": "Rahul Singh",
      "branch": "CSE",
      "cgpa": 8.75,
      "email": "rahul.singh@email.com"
    },
    {
      "id": 5,
      "rollNo": "9",
      "name": "Anita",
      "branch": "IT",
      "cgpa": 9.02,
      "email": "anita@email.com"
    }
  ]
}
```

Figure 1: Browser Output for GET /students Route

Supporting Files Used

The following files are used with this task:

- server.js — Express.js server application implementing routes and middleware.
- package.json — Project configuration and dependency manifest.

10. RESULT

Thus, the **Express.js Student Server** was successfully implemented, configured, and tested. The server provides the /, /students (serving 5 student records with query parameter filtering), and /about endpoints, handles unmapped paths with a 404 Not Found response, and incorporates request logging middleware.